

Shader Files

Column1	Shader Type	Input from previous shader	Uniform Input	Output	Small Description
light_vertex.glsl	Vertex	/	vec4 pos;		Sets the gl_position to the given pos.
light_frag.glsl	Fragment	/	vec3 color;	vec4 outColor;	Sets the outColor to the given color (alpha=1)
dot_vertex.glsl	Vertex	(layout) vec3 aPos;	mat4 mvp;	vec3 pos;	Set the gl_position and the pos output to the value of aPos (world vertex position from VBO) multiplied by the mvp matrix.
dot_frag.glsl	Fragment	vec3 pos;	int mode; vec3 cameraPos; sampler2D normalMap; sampler2D colorMap; sampler2D edgeMap; float renderSize; float renderHeight; float minDistanceToCamera; float maxDistanceToCamera;	vec4 outColor;	Outputs the dot color based on the rendering mode, the vertex position and the camera position. It creates a darkening gradient for the points based on the distance to the camera.
line_geometry.glsl	Geometry	layout(points)	mat4 mvp;	layout(line_strip, max_vertices = 2) vec3 pos;	Takes 2 vertices as input to form a line primitive, by applying the mvp matrix on each vertex.
quad_vertex.glsl	Vertex	(layout) vec3 aPos; vec2 aTexCoord;	(layout) mat4 mvp; float height;	vec2 TexCoord;	Set the gl_position to aPos + height, then multiplied by the mvp matrix. Outputs the TextCoord for the next shader.
quad_frag.glsl	Fragment	vec2 TexCoord;	sampler2D inputTexture;	vec4 outColor;	Outputs a color based on the texture coordinates and the input texture.
triangle_vertex.glsl	Vertex	(layout) vec3 aPos;	/	/	Sets the gl_position to the given pos.
triangle_geometry.glsl	Geometry	layout(lines_adjacency)	mat4 mvp;	layout(triangle_strip, max_vertices = 6) vec3 pos;	Takes 4 vertices as input, and based on a heuristic, returns 2 triangles. (multiply each vertex by the mvp matrix) The heuristic is simply to split the 4 vertices along the highest diagonal.
triangle_frag.glsl	Fragment	vec3 pos;	int lightAmount; uniform int mode; float renderHeight; sampler2D normalMap; sampler2D colorMap; sampler2D edgeMap; float renderSize; layout(std140) LightData { vec4 lightPos[20]; vec4 lightColor[20]; };	vec4 outColor;	Outputs the dot color based on the rendering mode, the vertex position and the light sources.
raytracing_vertex.glsl	Vertex	(layout) vec3 aPos; vec2 aTexCoord;	/	vec2 TexCoord;	Set the gl_position to aPos. Outputs the TextCoord for the next shader.

raytracing_frag.glsl	Fragment	vec2 TexCoord;	int lightAmount; uniform int mode; float renderHeight; sampler2D heightMap; sampler2D normalMap; sampler2D colorMap; sampler2D edgeMap; float renderSize; bool binarySearch; mat4 mvp; vec3 aabbMin; vec3 aabbMax; int maxSteps; layout(std140) LightData { vec4 lightPos[20]; vec4 lightColor[20]; };	vec4 FragColor;	Outputs the dot color based on the rendering mode, the screen position and the light sources. It traces a ray from the screen position through the AABB representing the terrain. The terrain is stored in a heightMap texture.
gaussian_blur.glsl	Fragment	vec2 TexCoord;	sampler2D inputTexture; vec2 direction; int size;	vec4 FragColor;	Outputs a color based on the texture, by applying a gaussian filter of the given size and direction on the original texture.
edge_detection_frag.glsl	Fragment	vec2 TexCoord;	sampler2D originalBlurredTexture; sampler2D largeBlurredTexture;	vec4 FragColor;	Outputs a color by subtracting the 2 input textures at the specified coordinates.

Shaders

	Stages	Description
lightShader	light_vertex.glsl → light_frag.glsl	Renders on-screen dots for each light.
dotShader	dot_vertex.glsl → dot_frag.glsl	Renders dots for each vertex in the data, based on the rendering mode.
quadShader	quad_vertex.glsl → quad_frag.glsl	Renders a flat image representing the data.
triangleShader	triangle_vertex.glsl → triangle_geometry.glsl → triangle_frag.glsl	Renders triangles to visualize the data, based on the rendering mode.
lineShader	triangle_vertex.glsl → line_geometry.glsl → dot_frag.glsl	Renders vertical lines for each vertex in the data.
raytracingShader	raytracing_vertex.glsl → raytracing_frag.glsl	Uses ray tracing to render the data, based on the rendering mode.
gaussianBlurShader	raytracing_vertex.glsl → gaussian_blur.glsl	Applies a gaussian blur on a texture and outputs the modified texture.
edgeDetectionShader	raytracing_vertex.glsl → edge_detection_frag.glsl	Creates a texture based on the subtraction of 2 blurred textures.
colormapVisualizationShader	raytracing_vertex.glsl → quad_frag.glsl	Renders a rectangle on the screen showing the color map used.